

18/04/2024

1

a) (1 punto)

¿En que dirección de memoria se encuentra la pila justo al empezar la ejecución de la rutina 'main' de sistema?

Es la dirección `0x3ff4`

¿Como la has encontrado?

```
$ make gdb
(gdb) b main          # breakpoint system main
(gdb) print $esp      # print stack addr
```

¿Corresponde a algún proceso del vector task?

No corresponde a ningún proceso del vector task, ya que todavía no se han ejecutado `set_eflags` ni `set_seg_regs` para cambiar `esp` a `&task[4]`

¿Cómo lo has hallado?

Continuando desde la ejecución anterior:

```
(gdb) b printk
(gdb) print $esp
$5 = (void *) 0x18fd8 <task+16344>
(gdb) print &task[4]
$6 = (union task_union *) 0x19000 <task+16384>
```

También, leyendo el código, vemos como se pasa el argumento `&task[4]` a la función `set_seg_regs`.

b) (1 punto) En el código inicial os damos el tratamiento de las excepciones ya implementado.

¿Es posible ver el código ensamblador de la rutina de tratamiento de la excepción de división por zero?

Sí.

Caso afirmativo muestra el código y cómo lo has hallado.

```

$ objdump -d libzeos.a | less
:/divide

00000210 <divide_error_routine>:
 210:  55                      push    %ebp
 211:  89 e5                   mov     %esp,%ebp
 213:  83 ec 14                sub     $0x14,%esp
 216:  68 00 00 00 00          push    $0x0
 21b:  e8 fc ff ff ff         call    21c <divide_error_routine+0xc>
 220:  c7 04 24 00 00 00 00    movl    $0x0,(%esp)
 227:  e8 fc ff ff ff         call    228 <divide_error_routine+0x18>
 22c:  83 c4 10                add     $0x10,%esp
 22f:  eb fe                  jmp     22f <divide_error_routine+0x1f>
 231:  8d b4 26 00 00 00 00    lea     0x0(%esi,%eiz,1),%esi
 238:  8d b4 26 00 00 00 00    lea     0x0(%esi,%eiz,1),%esi
 23f:  90                      nop

```

c) (1 punto) Dado un fichero exam.o que contiene la función shared implementada

¿Como debe ser la línea (o líneas) de comandos para linkar este objeto y que puedas usar la función tanto desde el código de sistema como de usuario?

Añadir al Makefile:

```

SYSOBJ = ... exam.o
USROBJ = exam.o

```

2

El bucle de copia de datos de usuario implica que hay que modificar entradas de la tabla de páginas (TP) del proceso actual temporalmente para mapear la zona de datos del proceso hijo. Para ahorrarnos esta modificación y dado que sólo estamos usando una entrada del directorio, queremos mapear temporalmente toda la TP del proceso hijo en la entrada 1 del directorio, y así realizar la copia de toda la zona de datos con una única llamada a copy_data. Implementa esta modificación del sys_fork.